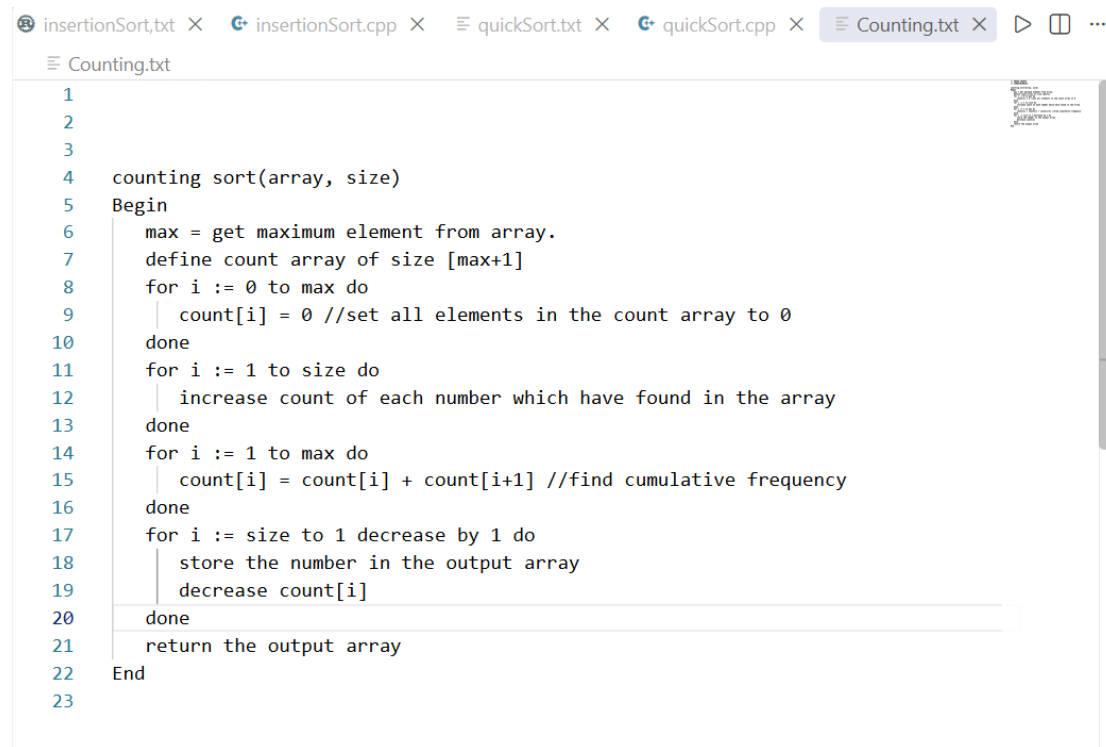


EXPERIMENT #4

Problem Statement: Write a program to sort an array using counting Sort.

Objectives: Counting sort is effective when range is not greater than number of objects to be sorted. It can be used to sort the negative input values.

Algorithm:



The image shows a screenshot of a code editor with multiple tabs. The active tab is 'Counting.txt'. The code is written in pseudocode for a counting sort algorithm. The code is as follows:

```
1
2
3
4 counting sort(array, size)
5 Begin
6     max = get maximum element from array.
7     define count array of size [max+1]
8     for i := 0 to max do
9         count[i] = 0 //set all elements in the count array to 0
10    done
11    for i := 1 to size do
12        increase count of each number which have found in the array
13    done
14    for i := 1 to max do
15        count[i] = count[i] + count[i+1] //find cumulative frequency
16    done
17    for i := size to 1 decrease by 1 do
18        store the number in the output array
19        decrease count[i]
20    done
21    return the output array
22 End
23
```

CODE:

```
insertionSort.txt × insertionSort.cpp × quickSort.txt × quickSort.cpp × Counting.txt × counting.cpp × HeapSort.cpp ×
counting.cpp > ...
1  ..
2
3
4 #include <stdio.h>
5 void countingSort(int arr[], int n) {
6     int i, max = arr[0];
7     // Find maximum value
8     for (i = 1; i < n; i++) {
9         if (arr[i] > max)
10            max = arr[i];
11    }
12    // Create count array
13    int count[max + 1];
14
15    // Initialize count array with 0
16    for (i = 0; i <= max; i++)
17        count[i] = 0;
18
19    // Store the count of each element
20    for (i = 0; i < n; i++)
21        count[arr[i]]++;
22
23    // Modify count array (cumulative sum)
24    for (i = 1; i <= max; i++)
25        count[i] += count[i - 1];
26
27    // Build output array
28    int output[n];
29    for (i = n - 1; i >= 0; i--) {
30        output[count[arr[i]] - 1] = arr[i];
31        count[arr[i]]--;
32    }
33    // Copy output to original array
34    for (i = 0; i < n; i++)
35        arr[i] = output[i];
36 }
37 int main() {
38     int arr[] = {4, 9, 2, 8, 3, 6, 1};
39     int n = sizeof(arr) / sizeof(arr[0]);
40     countingSort(arr, n);
41     printf("Sorted array: ");
42     for (int i = 0; i < n; i++)
43         printf("%d ", arr[i]);
44     return 0;
45 }
```

OUTPUT:

```
Problems Output Debug Console Terminal Ports
● PS C:\Users\anshu\Documents\DAA> cd "c:\Users\anshu\Documents\DAA\" ; if ($?) { g++ counting.cpp -o counting } ; if ($?) { .\co
unting }
Sorted array: 1 2 3 4 6 8 9
○ PS C:\Users\anshu\Documents\DAA> []
```

EXPERIMENT : #5

Problem Statement: Write a program to sort an array using HeapSort.

Objectives: Heapsort is an efficient sorting algorithm with $O(n \log n)$ worst case time complexity. It is an inplace sorting algorithm.

Algorithm:

HEAPSORT(*A*)

```
1  BUILD-MAX-HEAP(A)
2  for i = A.length downto 2
3      exchange A[1] with A[i]
4      A.heap-size = A.heap-size - 1
5      MAX-HEAPIFY(A, 1)
```

BUILD-MAX-HEAP(*A*)

```
1  A.heap-size = A.length
2  for i =  $\lfloor A.length/2 \rfloor$  downto 1
3      MAX-HEAPIFY(A, i)
```

MAX-HEAPIFY(*A*, *i*)

```
1  l = LEFT(i)
2  r = RIGHT(i)
3  if  $l \leq A.heap-size$  and  $A[l] > A[i]$ 
4      largest = l
5  else largest = i
6  if  $r \leq A.heap-size$  and  $A[r] > A[largest]$ 
7      largest = r
8  if largest  $\neq i$ 
9      exchange A[i] with A[largest]
10     MAX-HEAPIFY(A, largest)
```

PARENT(*i*)

```
1  return  $\lfloor i/2 \rfloor$ 
```

LEFT(*i*)

```
1  return  $2i$ 
```

RIGHT(*i*)

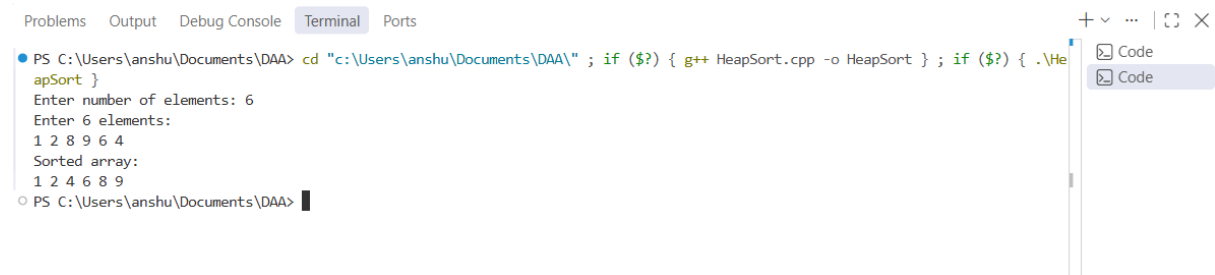
```
1  return  $2i + 1$ 
```

CODE :

```
insertionSort.txt x insertionSort.cpp x quickSort.txt x quickSort.cpp x Counting.txt x counting.cpp x HeapSort.cpp x
HeapSort.cpp > ...
1
2
3
4 #include <iostream>
5 #include <vector>
6
7 using namespace std;
8
9 // Helper function to get left child index (1-based indexing)
10 int LEFT(int i) {
11     return 2 * i;
12 }
13 // Helper function to get right child index (1-based indexing)
14 int RIGHT(int i) {
15     return 2 * i + 1;
16 }
17 // MAX-HEAPIFY as described in the manual
18 void MAX_HEAPIFY(vector<int>& A, int i, int heap_size) {
19     int l = LEFT(i);
20     int r = RIGHT(i);
21     int largest = i;
22     // Compare with left child
23     if (l <= heap_size && A[l] > A[i]) {
24         largest = l;
25     } else {
26         largest = i;
27     }
28     // Compare with right child
29     if (r <= heap_size && A[r] > A[largest]) {
30         largest = r;
31     }
32     // If largest is not root
33     if (largest != i) {
34         swap(A[i], A[largest]);
35         MAX_HEAPIFY(A, largest, heap_size);
36     }
37 }
38 // BUILD-MAX-HEAP procedure
39 void BUILD_MAX_HEAP(vector<int>& A, int n) {
40     for (int i = n / 2; i >= 1; i--) {
41         MAX_HEAPIFY(A, i, n);
42     }
43 }
44 // HEAPSORT procedure as described in the manual
45 void HEAPSORT(vector<int>& A, int n) {
46     BUILD_MAX_HEAP(A, n);
```

```
insertionSort.txt x insertionSort.cpp x quickSort.txt x quickSort.cpp x Counting.txt x counting.cpp x HeapSort.cpp x
HeapSort.cpp > main()
42 void HEAPSORT(vector<int>& A, int n) {
44     int heap_size = n;
45     for (int i = n; i >= 2; i--) {
46         swap(A[1], A[i]);
47         heap_size = heap_size - 1;
48         MAX_HEAPIFY(A, 1, heap_size);
49     }
50 }
51 int main() {
52     int n;
53     cout << "Enter number of elements: ";
54     if (!(cin >> n) || n <= 0) return 0;
55
56     // Using 1-based indexing: allocate n + 1 elements
57     vector<int> A(n + 1);
58     cout << "Enter " << n << " elements:\n";
59     for (int i = 1; i <= n; i++) {
60         cin >> A[i];
61     }
62
63     HEAPSORT(A, n);
64
65     cout << "Sorted array:\n";
66     for (int i = 1; i <= n; i++) {
67         cout << A[i] << " ";
68     }
69     cout << endl;
70
71     return 0;
72 }
```

OUTPUT :



```
PS C:\Users\anshu\Documents\DAA> cd "c:\Users\anshu\Documents\DAA\" ; if ($?) { g++ HeapSort.cpp -o HeapSort } ; if ($?) { .\HeapSort }
Enter number of elements: 6
Enter 6 elements:
1 2 8 9 6 4
Sorted array:
1 2 4 6 8 9
PS C:\Users\anshu\Documents\DAA>
```

EXPERIMENT : #1

Problem Statement: Write a Program to sort an array using Insertion sort.

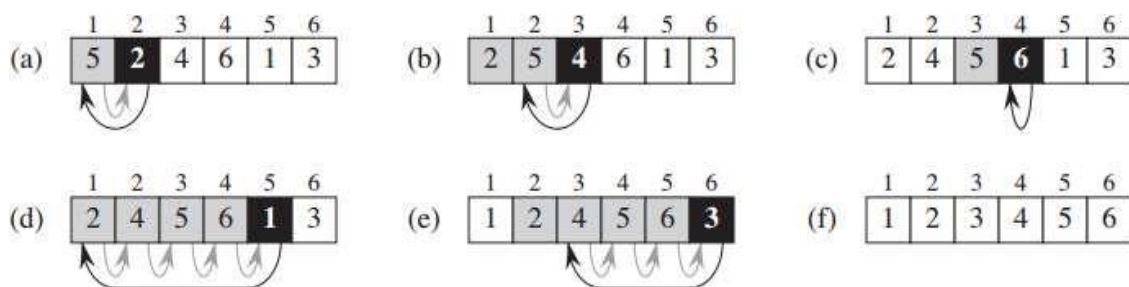
Objectives: The insertion sort is useful when the array is nearly sorted or the array size is small. When humans manually sort something (for example, a deck of playing cards), most use a method that is similar to insertion sort.

Inputs: An array A[]

Algorithm:

```
mergeSort.cpp × insertionSort.txt × insertionSort.cpp × mergeSort.txt ×
insertionSort.txt
1
2
3
4  INSERTION-SORT(A)
5  for j = 2 to length[A]
6      do key = A[j]
7      // Insert A[j] into the sorted sequence A[1..j-1]
8      i = j - 1
9      while i > 0 and A[i] > key
10         do A[i + 1] = A[i]
11         i = i - 1
12     A[i + 1] = key
```

Example:



CODE:

```
insertionSort.txt x insertionSort.cpp x quickSort.txt x quickSort.cpp x Counting.txt x counting.cpp x HeapSort.cpp x
insertionSort.cpp > ...
1
2
3
4 #include <iostream>
5 using namespace std;
6
7 void insertionSort(int A[], int n) {
8     for (int j = 1; j < n; j++) {
9         int key = A[j];
10        int i = j - 1;
11        while (i >= 0 && A[i] > key) {
12            A[i + 1] = A[i];
13            i = i - 1;
14        }
15        A[i + 1] = key;
16    }
17 }
18 int main() {
19     int n;
20     cout << "Enter number of elements: ";
21     cin >> n;
22
23     int A[100];
24     cout << "Enter elements: ";
25     for (int i = 0; i < n; i++) {
26         cin >> A[i];
27     }
28     insertionSort(A, n);
29
30     cout << "Sorted array: ";
31     for (int i = 0; i < n; i++) {
32         cout << A[i] << " ";
33     }
34     cout << endl;
35     return 0;
36 }
```

Output :

```
Problems Output Debug Console Terminal Ports
cd "c:\Users\anshu\Documents\DAA\" ; if ($?) { g++ insertionSort.cpp -o insertionSort } ; if ($?) { .\insertionSort }
Sort }
Enter number of elements: 6
Enter elements: 5 2 4 6 1 3
Sorted array: 1 2 3 4 5 6
PS C:\Users\anshu\Documents\DAA>
```

EXPERIMENT : #3

Problem Statement: Write a program to sort an array using Merge Sort.

Objectives: Merge Sort is good for sorting slow-access data. It is excellent for sorting data that normally accessed sequentially eg. linked list, tape drive, hard disk and receiving online data one at a time.

Algorithm:

```
mergeSort.cpp X mergeSor.txt X
mergeSor.txt
1
2
3
4  MERGE-SORT
5  If p<r
6      q[(p+r)/2]
7      MERGESORT(A,p,q)
8      MERGESORT(A,q+1,r)
9      MERGE(A,p,q,r)
10
11  MERGE(A,p,q,r)
12  n1 = q-p+1
13  n2 = r-q
14  let L[1..n1+1] and R[1..n2+1] be new arrays
15  for i = 1 to n1
16      L[i] = A[p+i-1]
17  for j=1 to n2
18      R[j] = A[q+j]
19  L[n1+1] = ∞
20  R[n2+1] = ∞
21  i=1
22  j=1
23  for k = p to r
24      if L[i] <= R[j]
25          A[k] = L[i]
26          i = i+1
27      else A[k] = R[j]
28          j = j+1
```


CODE:

```
mergeSort.cpp X insertionSort.txt X insertionSort.cpp X quickSort.txt X quickSort.cpp X Counting.txt X counting.cp
mergeSort.cpp > ...
1
2
3
4 #include <iostream>
5 using namespace std;
6 void merge(int arr[], int low, int mid, int high) {
7     int i = low;
8     int j = mid + 1;
9     int k = 0;
10    int temp[100];
11    // Compare and merge two sorted parts
12    while (i <= mid && j <= high) {
13        if (arr[i] <= arr[j]) {
14            temp[k] = arr[i];
15            i++;
16        } else {
17            temp[k] = arr[j];
18            j++;
19        }
20        k++;
21    }
22    // Copy remaining elements from left part
23    while (i <= mid) {
24        temp[k] = arr[i];
25        i++;
26        k++;
27    }
28    // Copy remaining elements from right part
29    while (j <= high) {
30        temp[k] = arr[j];
31        j++;
32        k++;
33    }
34    // Copy sorted elements back to original array
35    for (i = low, k = 0; i <= high; i++, k++) {
36        arr[i] = temp[k];
37    }
38 }
39 void mergeSort(int arr[], int low, int high) {
40     if (low < high) {
41         int mid = (low + high) / 2;
42
43         // Divide
44         mergeSort(arr, low, mid);
45         mergeSort(arr, mid + 1, high);
46
47         // Conquer/Merge
48         merge(arr, low, mid, high);
49     }
50 }
51 int main() {
52     int n;
53     cout << "Enter number of elements: ";
54     cin >> n;
55     int arr[100];
56     cout << "Enter elements: ";
57     for (int i = 0; i < n; i++) {
58         cin >> arr[i];
59     }
60     mergeSort(arr, 0, n - 1);
61     cout << "Sorted array: ";
62     for (int i = 0; i < n; i++) {
63         cout << arr[i] << " ";
64     }
65     return 0;
66 }
```

OUTPUT:

```
Problems Output Debug Console Terminal Ports
PS C:\Users\anshu\Documents\DAA>

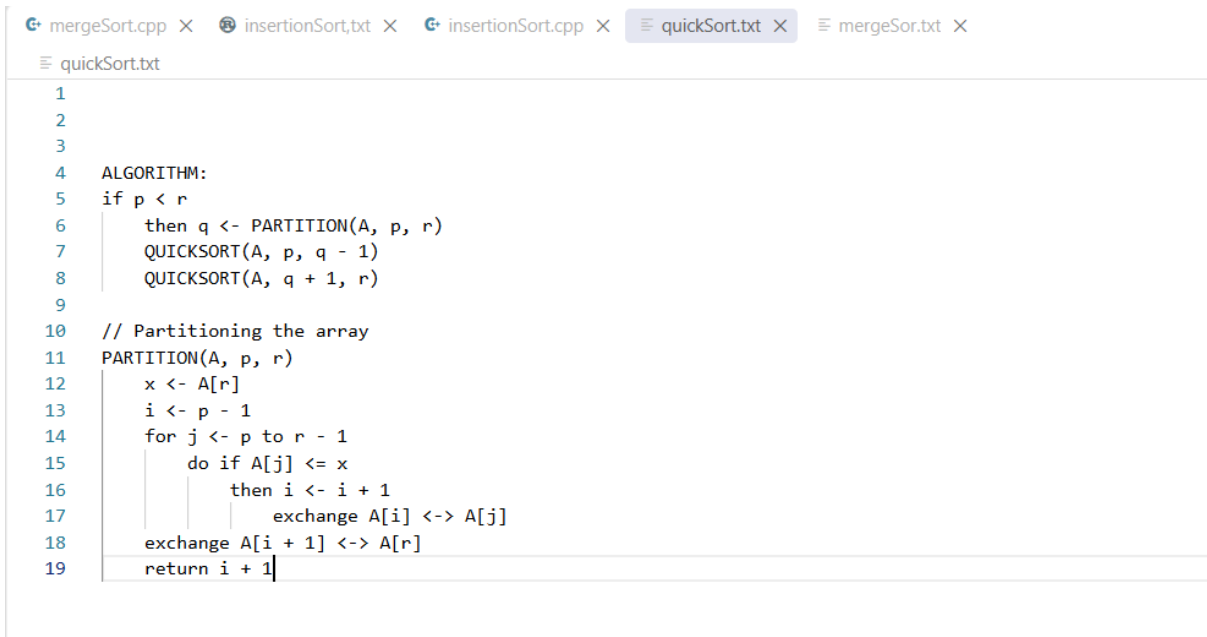
cd "c:\Users\anshu\Documents\DAA\" ; if ($?) { g++ mergeSort.cpp -o mergeSort } ; if ($?) { .\mergeSort }
Enter number of elements: 6
Enter elements: 1 3 5 7 6 8 9
Sorted array: 1 3 5 6 7 8
PS C:\Users\anshu\Documents\DAA>
```

EXPERIMENT : #2

Problem Statement: Write a program to sort an array using QuickSort.

Objectives: Quicksort is an inplace sorting algorithm, It is brilliant as it's inner loop can be efficiently implemented on most computer architectures.

Algorithm:



```
mergeSort.cpp X insertionSort.txt X insertionSort.cpp X quickSort.txt X mergeSort.txt X
quickSort.txt
1
2
3
4 ALGORITHM:
5 if p < r
6   then q <- PARTITION(A, p, r)
7   QUICKSORT(A, p, q - 1)
8   QUICKSORT(A, q + 1, r)
9
10 // Partitioning the array
11 PARTITION(A, p, r)
12   x <- A[r]
13   i <- p - 1
14   for j <- p to r - 1
15     do if A[j] <= x
16       then i <- i + 1
17       exchange A[i] <-> A[j]
18   exchange A[i + 1] <-> A[r]
19   return i + 1
```

CODE:

```
mergeSort.cpp X insertionSort.txt X insertionSort.cpp X quickSort.txt X quickSort.cpp X Counting.txt X counting.cpp >
quickSort.cpp > ...
1
2
3
4 #include <iostream>
5 using namespace std;
6 // Function to partition the array
7 int partition(int arr[], int low, int high) {
8     int pivot = arr[high]; // Choose last element as pivot
9     int i = low - 1;
10
11     for (int j = low; j < high; j++) {
12         if (arr[j] < pivot) {
13             i++;
14             swap(arr[i], arr[j]);
15         }
16     }
17     swap(arr[i + 1], arr[high]);
18     return i + 1;
19 }
20
21 // Quick Sort function
22 void quickSort(int arr[], int low, int high) {
23     if (low < high) {
24         int pi = partition(arr, low, high);
25         quickSort(arr, low, pi - 1); // Left subarray
26         quickSort(arr, pi + 1, high); // Right subarray
27     }
28 }
29
30 // Function to print array
31 void printArray(int arr[], int size) {
32     for (int i = 0; i < size; i++) {
33         cout << arr[i] << " ";
34     }
35     cout << endl;
36 }
37
38 int main() {
39     int arr[] = {10, 7, 8, 9, 1, 5};
40     int n = sizeof(arr) / sizeof(arr[0]);
41
42     cout << "Original array: ";
43     printArray(arr, n);
44
45     quickSort(arr, 0, n - 1);
46
47     cout << "Sorted array: ";
48     printArray(arr, n);
49
50     return 0;
51 }
```

OUTPUT :

```
Problems Output Debug Console Terminal Ports
Code + v [ ] [ ] ... [ ] [ ] X

cd "c:\Users\anshu\Documents\DAA\" ; if ($?) { g++ quickSort.cpp -o quickSort } ; if ($?) { .\quickSort }
Original array: 10 7 8 9 1 5
Sorted array: 1 5 7 8 9 10
PS C:\Users\anshu\Documents\DAA> 
```

